

MTL766 Report
Dimensionality Reduction Methods - UMAP, kPCA,
tSNE

Saem Habeeb	2022MT62004
Suhani Soni	2022MT61981
Dhruv Chaurasiya	2022MT11172

October 19, 2025

Contents

1	Overview	3
2	UMAP	5
2.1	Introduction: From Geometric Intuition to Topological Representation . . .	5
2.2	The Manifold Hypothesis: Approximating Local Structure	8
2.2.1	What is a Manifold?	8
2.2.2	UMAP’s Simplifying Assumptions	9
2.2.3	From Assumptions to a Local Metric	10
2.2.4	Finding the Scaling Factor (σ_i)	11
2.2.5	The Consequence: Asymmetry and the Need for Topology	11
2.3	Mathematical Preliminaries: Tools for Topological Modeling	12
2.3.1	Simplicial Complexes and Simplicial Sets	12
2.3.2	A Primer on Category Theory: The Language of Translation	14
2.3.3	Fuzzy Set Theory	15
2.3.4	Fuzzy Simplicial Sets	15
2.4	Constructing the High-Dimensional Fuzzy Topological Representation	16
2.4.1	The Problem: A World of Asymmetric “Distances”	16
2.4.2	The Solution: A Principled “Translation”	17
2.4.3	Step 1: Translate Local Distances into Local Probabilities	17
2.4.4	Step 2: “Stitch” Local Views into a Global Whole	18
2.5	Optimization: Finding the Low-Dimensional Embedding	18
2.5.1	Step 1: Building the Low-Dimensional Fuzzy Set	18
2.5.2	Step 2: Comparing the High-D and Low-D Structures	19
2.5.3	Step 3: The “Force-Directed” Layout	19
2.6	From Theory to Practice: The UMAP Algorithm	20
2.6.1	The Initialization Problem: Avoiding Bad Starts	21
2.6.2	Spectral Initialization: Using the Graph’s “Vibration”	21
2.6.3	Tuning the Algorithm: Key Hyperparameters	22
2.7	Empirical Evaluation and Conclusion	23
2.7.1	Qualitative Evaluation	23
2.7.2	Quantitative Evaluation	24
2.7.3	Conclusion	24

3	kPCA	26
3.1	PCA	26
3.1.1	Data and notation	26
3.1.2	PCA formulation and results	27
3.1.3	Why kPCA?	28
3.1.4	Kernels	29
3.2	kPCA	30
3.2.1	Data and Notation	30
3.2.2	kPCA Algorithm	31
4	SNE	32
4.1	SNE	32
4.2	Symmetric SNE	33
4.3	t-SNE	34
4.4	kernel t-SNE	35

Chapter 1

Overview

The rapid proliferation of high-dimensional data across scientific, financial, and technological domains has presented a fundamental challenge to data analysis. In high-dimensional spaces, data often becomes sparse, distance metrics lose their intuitive meaning, and direct visualization is impossible—a phenomenon collectively known as the **curse of dimensionality**. Dimensionality reduction techniques are essential tools designed to combat this problem by creating a faithful, low-dimensional representation of the data that preserves its intrinsic structure, facilitating visualization, noise reduction, and subsequent machine learning tasks.

While classical linear methods, such as Principal Component Analysis (PCA), are computationally efficient and effective at capturing the global variance of data, they fundamentally fail when the data exhibits complex, **non-linear** patterns. Many real-world datasets are better described as lying on or near a low-dimensional, curved **manifold** embedded within the high-dimensional ambient space.

This report provides a detailed exploration of several powerful **non-linear dimensionality reduction** techniques that have been developed to address this challenge.

This report is structured as follows:

- **Chapter 2: Uniform Manifold Approximation and Projection (UMAP).** We begin with a deep dive into UMAP, a state-of-the-art manifold learning algorithm. UMAP is founded on a rigorous mathematical framework of Riemannian geometry and algebraic topology. We will explore how it constructs a high-dimensional topological representation (a fuzzy simplicial set) and then optimizes a low-dimensional embedding to be as topologically similar as possible.
- **Chapter 3: Kernel Principal Component Analysis (kPCA).** This chapter details kPCA, a direct non-linear generalization of classical PCA. We will first review the mathematical foundations of PCA and then demonstrate how the kernel trick is applied to generalize this method, allowing it to find non-linear principal components in a high-dimensional feature space.
- **Chapter 4: Stochastic Neighbor Embedding (SNE).** Finally, we explore a family of probabilistic methods. We begin with the foundational SNE, which models pairwise similarities as conditional probabilities. We then discuss its popular and more robust

successor, **t-Distributed Stochastic Neighbor Embedding (t-SNE)**, which famously resolves the “crowding problem.” We conclude by examining **Kernel t-SNE**, which integrates the kernel concept to modify the distance metric used in the algorithm.

Through a detailed review of their mathematical derivations and core concepts, this report aims to provide a clear and comprehensive understanding of these advanced and widely used dimensionality reduction methods.

Chapter 2

UMAP

2.1 Introduction: From Geometric Intuition to Topological Representation

The proliferation of high-dimensional data across scientific and industrial domains has established dimensionality reduction as an indispensable tool in the modern data analysis pipeline. The primary objective of any such technique is to produce a faithful, low-dimensional representation of a high-dimensional dataset, simplifying visualization, mitigating the curse of dimensionality, and facilitating subsequent machine learning tasks. [3][4]

However, the notion of a “faithful” representation is not uniquely defined and has led to a spectrum of algorithms, each optimizing for different aspects of data structure.

At one end of this spectrum lie linear methods such as **Principal Component Analysis (PCA)**, which excel at preserving the global variance structure of the data by projecting it onto a set of orthogonal axes that capture maximal variance. While computationally efficient and highly interpretable, PCA’s linearity fundamentally limits its ability to represent complex, non-linear relationships inherent in many real-world datasets.

At the other end are non-linear manifold learning techniques, which operate under the assumption that the data lies on or near a low-dimensional manifold embedded within the high-dimensional ambient space. Algorithms in this class, such as **t-Distributed Stochastic Neighbor Embedding (t-SNE)**, prioritize the preservation of local neighborhood structures. t-SNE, in particular, has become a de facto standard for data visualization, renowned for its ability to create well-separated clusters that reveal fine-grained local similarities. It achieves this by modeling pairwise similarities as conditional probabilities and minimizing the Kullback-Leibler (KL) divergence between the probability distributions of the high- and low-dimensional spaces.

Despite its success, t-SNE’s focus on local structure often comes at the expense of capturing the data’s global topology, and its probabilistic formulation includes a computationally intensive normalization step that limits its scalability to large datasets.

We now look at **Uniform Manifold Approximation and Projection (UMAP)**, a novel manifold learning technique that seeks to overcome the limitations of its predecessors by fundamentally reframing the dimensionality reduction problem. UMAP represents a paradigm shift away from the probabilistic modeling of t-SNE towards a framework grounded

in Riemannian geometry and algebraic topology.

Its central thesis is that a more faithful representation can be achieved by first constructing a topological model of the data—specifically, a fuzzy simplicial set—and then optimizing a low-dimensional embedding to match this topological structure as closely as possible.

This topological approach provides a robust theoretical foundation that leads to several practical advantages. As demonstrated in Figure 2.1, UMAP often produces visualizations that are not only competitive with t-SNE in revealing local cluster structure but also superior in preserving the global relationships between those clusters.

Furthermore, the mathematical framework allows for a highly efficient optimization process, resulting in significantly better runtime performance and scalability. The remainder of this report will deconstruct the UMAP algorithm from first principles, beginning with its foundational geometric assumptions, building the necessary mathematical machinery from topology and category theory, deriving the high-dimensional topological construction and low-dimensional optimization, and concluding with an empirical evaluation of its performance.

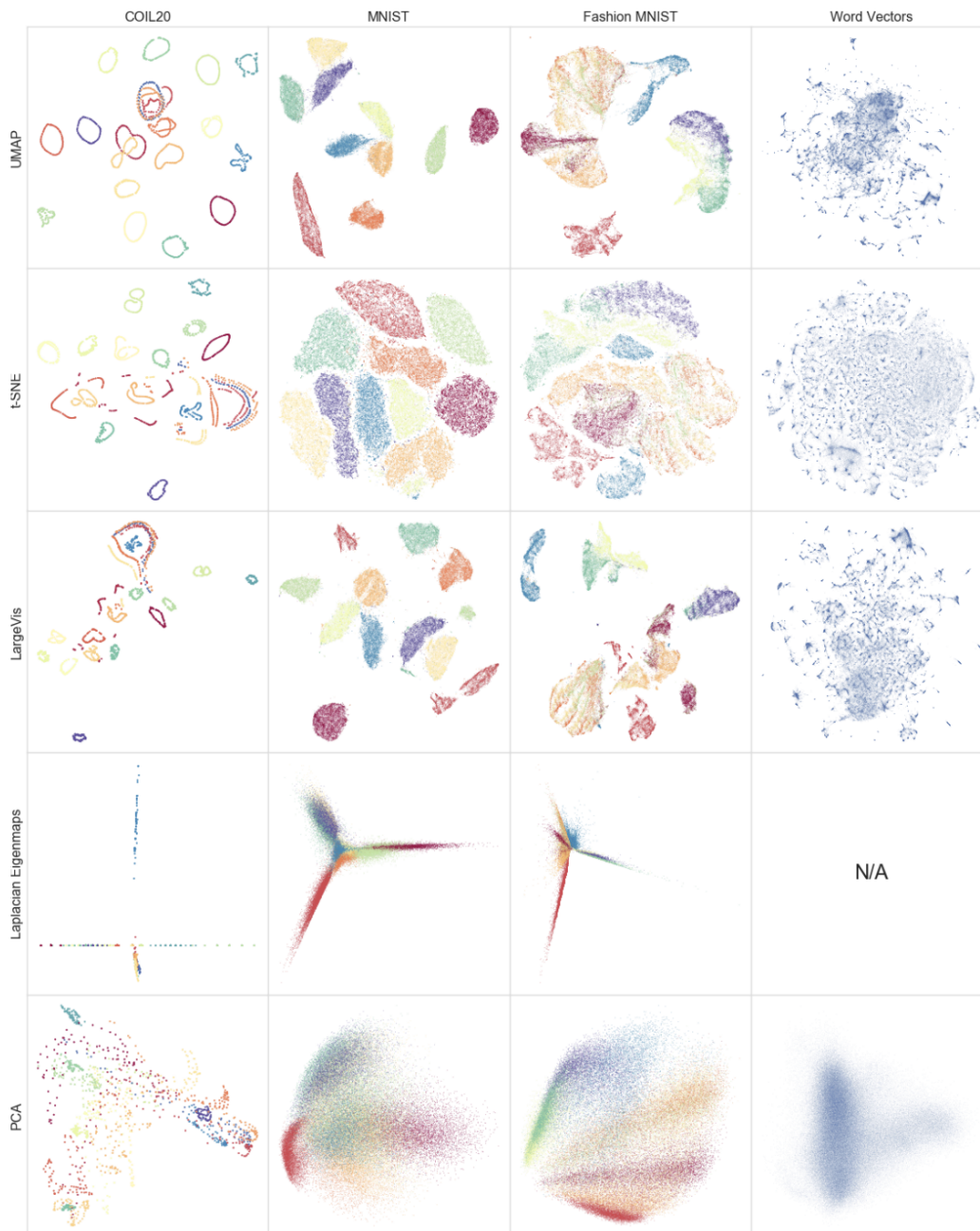


Figure 2.1: *A comparison of several dimension reduction algorithms. UMAP is shown to capture both the local cluster structure characteristic of t-SNE and LargeVis, as well as the broader global structure often revealed by methods like PCA and Laplacian Eigenmaps.*

2.2 The Manifold Hypothesis: Approximating Local Structure

The theoretical underpinnings of UMAP, and indeed most non-linear dimensionality reduction techniques, rest upon the **manifold hypothesis**.

This hypothesis posits that a high-dimensional dataset $D = \{x_1, x_2, \dots, x_N\}$ with $x_i \in \mathbb{R}^N$ does not populate the ambient space uniformly, but rather lies on or near a low-dimensional Riemannian manifold $\mathcal{M}$ of intrinsic dimension $m \ll N$.

This manifold is assumed to be embedded in the high-dimensional space via some (unknown) injective map $\phi : \mathcal{M} \hookrightarrow \mathbb{R}^N$.

Figure 2.2 provides a conceptual illustration of this setup. The goal of manifold learning is to effectively “unroll” this manifold to find a coordinate representation in a lower-dimensional space, $\mathbb{R}^m$, that preserves its intrinsic geometric properties.

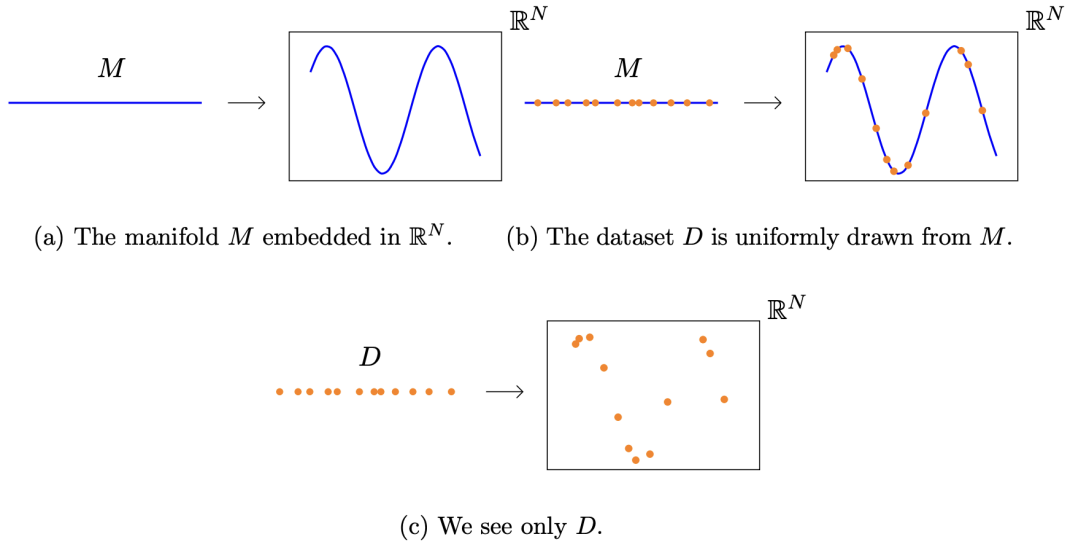


Figure 2.2: **A conceptual illustration of the manifold hypothesis.** (a) A low-dimensional manifold $\mathcal{M}$ is embedded in a high-dimensional space $\mathbb{R}^N$. (b) The observed dataset D consists of points sampled from this manifold. (c) The algorithm only has access to the points in D and must infer the structure of $\mathcal{M}$ from them.

The core idea behind UMAP is that high-dimensional data, like images or text embeddings, doesn’t actually fill the entire high-dimensional space. Instead, it is assumed to lie on or near a much lower-dimensional structure “curved” or “embedded” within the higher-dimensional space. This structure is known as a manifold.

2.2.1 What is a Manifold?

Imagine a flat, 2D sheet of paper. This is a simple 2D Euclidean space. Now, crumple that sheet of paper into a 3D ball. The paper itself is still fundamentally 2D—an ant walking on

its surface only ever experiences a 2D world. At any tiny spot, the paper looks like a flat 2D plane.

This crumpled paper is an example of a 2D **manifold** ($\mathcal{M}$) embedded in a 3D **ambient space** ($\mathbb{R}^3$). UMAP generalizes this: our observed data lives in a high-dimensional ambient space $\mathbb{R}^N$, but we assume it’s “drawn” on a much lower-dimensional manifold $\mathcal{M}$ (with intrinsic dimension m , where $m \ll N$).

However, the “crumpling” of the manifold changes how we measure distances. The straight-line distance between two points in the 3D space (drilling through the air) is the **ambient distance** ($d_{\mathbb{R}^N}$). The “ant-walking” distance along the paper’s surface is the true **geodesic distance** ($d_{\mathcal{M}}$). This intrinsic geometry is what UMAP seeks to preserve.

To do this, we must formally define this intrinsic geometry. This is done by adding a “ruler” to the manifold.

A **Riemannian manifold** $(\mathcal{M}, g)$ is a real, differentiable manifold $\mathcal{M}$ equipped with a smoothly varying inner product g_p on the tangent space $T_p\mathcal{M}$ at each point $p \in \mathcal{M}$.

Let’s break this down:

- **Tangent Space** ($T_p\mathcal{M}$): At any point p on our crumpled paper, this is the small, flat 2D plane that just touches the surface at that single point. It represents all possible “directions” the ant can walk from point p .
- **Inner Product** (g_p): This is a “local ruler” defined on that tangent space. It’s a function that tells us how to measure lengths of vectors (directions) and the angles between them at that specific point p .
- **Smoothly Varying**: This just means the “crumpling” is smooth, not a sharp, infinite fold. The local ruler g_p changes gradually as we move from p to a nearby point q .

This inner product g_p is also called the **Riemannian metric** or **metric tensor**. It is the mathematical tool that defines the manifold’s intrinsic geometry, allowing us to compute the “true” geodesic distances, angles, and volumes along the manifold’s surface.

2.2.2 UMAP’s Simplifying Assumptions

The central problem is that we *only* have the data points in the ambient space $\mathbb{R}^N$. We do not know the manifold $\mathcal{M}$ or its metric g . To infer the geometry from the data, UMAP makes three powerful simplifying assumptions:

1. The data is **uniformly distributed** on the manifold $\mathcal{M}$.
2. The Riemannian metric g is **locally constant**.
3. The manifold $\mathcal{M}$ is **locally connected**.

Assumption 1: Uniform Distribution This is the “Uniform” in UMAP. It assumes that our data points were spread out evenly across the manifold $\mathcal{M}$ *before* it was embedded (or “crumpled”) into $\mathbb{R}^N$.

The varying densities we *observe* in the ambient space $\mathbb{R}^N$ are just an illusion caused by the embedding.

- **Dense regions** in $\mathbb{R}^N$ correspond to areas where the manifold was “compressed.”
- **Sparse regions** in $\mathbb{R}^N$ correspond to areas where the manifold was “stretched.”

This assumption is a powerful theoretical tool. It implies that any two regions of equal volume *on the manifold* should contain approximately the same number of data points. This allows UMAP to “invert the logic”: instead of fixing a volume and counting points, it fixes the number of points (k) and infers the local volume.

Assumption 2: Locally Constant Metric This assumption states that within any small-enough patch on the manifold, the metric g doesn’t change. In our analogy, the “stretching factor” is constant within that small patch.

The crucial implication is that if the metric is constant, the patch is “flat.” This means the intrinsic geodesic distances $d_{\mathcal{M}}$ within that patch are just a *scaled version* of the ambient Euclidean distances $d_{\mathbb{R}^N}$. The manifold’s local geometry is not arbitrarily complex, it just behaves like a piece of $\mathbb{R}^m$ that has been uniformly scaled and embedded in $\mathbb{R}^N$.

This assumption is what allows us to relate the unknown manifold distances to the computable ambient distances.

Assumption 3: Locally Connected This is a simpler assumption: the manifold does not have isolated points or tiny, disconnected “islands.” Every point has some neighbors. This assumption is what justifies the use of k -nearest neighbors in the first place and is practically enforced by using ρ_i , the distance to the *first* nearest neighbor, as a baseline.

2.2.3 From Assumptions to a Local Metric

The assumptions of a locally constant metric and a uniform data distribution lead to a cornerstone lemma that justifies UMAP’s algorithmic approach.

Lemma 1. Let $(\mathcal{M}, g)$ be a Riemannian manifold embedded in $\mathbb{R}^N$. Let $p \in \mathcal{M}$ be a point. Suppose that g is locally constant. Let B be a ball in $\mathcal{M}$, containing p , whose volume is $\frac{\pi^{m/2}}{\Gamma(m/2+1)}$ with respect to the metric on $\mathcal{M}$. Then the distance of the shortest path in $\mathcal{M}$ from p to a point $q \in B$ is $\frac{1}{r}d_{\mathbb{R}^N}(p, q)$, where r is the radius of B in $\mathbb{R}^N$ and $d_{\mathbb{R}^N}(p, q)$ is the distance from p to q in $\mathbb{R}^N$.

Understanding Lemma 1 This lemma is dense but its consequence is simple and profound.

- “a ball... whose volume is...”: The term $\frac{\pi^{m/2}}{\Gamma(m/2+1)}$ is just the volume of a standard **unit m -ball** (a ball of radius 1 in $\mathbb{R}^m$). By the uniformity assumption, we can “fix” the number of points (k) and assume this corresponds to a patch of standard, constant volume on the manifold.
- “the distance... is $\frac{1}{r}d_{\mathbb{R}^N}(p, q)$ ”: It gives us a formula:

$$d_{\mathcal{M}}(p, q) = \frac{1}{r}d_{\mathbb{R}^N}(p, q)$$

This means the “true” geodesic distance ($d_{\mathcal{M}}$) is just the “observed” ambient distance ($d_{\mathbb{R}^N}$) divided by a scaling factor r .

- What is r ? It’s the radius of the k -neighbor ball in the *ambient space* $\mathbb{R}^N$.
 - In a **sparse region**, we must go “far” in $\mathbb{R}^N$ to find k neighbors, so r will be **large**. The manifold is “stretched” here. We must divide by a large r to “un-stretch” the distances.
 - In a **dense region**, the k neighbors are very close in $\mathbb{R}^N$, so r will be **small**. The manifold is “compressed.” We must divide by a small r to “un-compress” the distances.

This lemma provides a direct link between the unknown geodesic distances and the computable Euclidean distances, scaled by a local, density-dependent factor r .

2.2.4 Finding the Scaling Factor (σ_i)

In practice, we don’t have a perfectly defined radius r . Instead, UMAP computes a “fuzzy” or probabilistic version of this local scaling factor, σ_i , for *each data point* x_i .

For each point x_i , the algorithm finds its k nearest neighbors $\{x_{i_1}, \dots, x_{i_k}\}$. It then seeks a scaling factor σ_i (analogous to r) that “normalizes” this local neighborhood. This σ_i is found by solving the following equation for a fixed k :

$$\sum_{j=1}^k \exp\left(\frac{-\max(0, d(x_i, x_{i_j}) - \rho_i)}{\sigma_i}\right) = \log_2(k) \quad (2.1)$$

Here’s what this equation means:

- $\rho_i = d(x_i, x_{i_1})$ is the distance to x_i ’s **nearest** neighbor. The $\max(0, \dots - \rho_i)$ term ensures local connectivity by effectively “connecting” all points up to ρ_i with a maximum weight of 1.
- The $\exp(\dots)$ term is a “fuzzy” measure of similarity. It creates a soft, weighted graph of neighbors, where the “steepness” of the decay is controlled by σ_i .
- $\log_2(k)$ is the Shannon entropy of a uniform distribution over k choices. It represents the “effective number of neighbors” we want our fuzzy neighborhood to have.

This equation is solved (typically via binary search) for σ_i for *every single data point*. This σ_i is a unique, locally-adapted metric scaling factor for the neighborhood of x_i .

2.2.5 The Consequence: Asymmetry and the Need for Topology

This construction—finding a unique σ_i for each point x_i —is a direct consequence of the manifold assumptions, but it introduces a fundamental challenge.

The scaling factor σ_i depends on the data density around x_i . Its neighbor, x_j , is in a different location and will have a different local density, so it will have its own scaling factor, σ_j .

Since local densities are generally different, $\sigma_i \neq \sigma_j$.

This means the “fuzzy distance” or “similarity” from x_i to x_j (calculated using σ_i) is **not equal** to the similarity from x_j to x_i (calculated using σ_j).

$$w(x_i \rightarrow x_j) \neq w(x_j \rightarrow x_i)$$

This asymmetry is not a flaw; it is a direct and necessary consequence of correctly adapting the metric to local data density. However, it means we no longer have a single, simple “metric space” where the distance from A to B is the same as from B to A.

Instead, we have a family of N incompatible, locally-defined metric spaces—one centered at each of the N data points. The central problem of UMAP then becomes: **How do we reconcile, or “stitch together,” these N disparate, asymmetric local views into a single, globally consistent topological structure?**

This necessity forces a move away from the familiar world of metric spaces and into a more abstract mathematical framework capable of handling such inconsistencies, leading directly to the use of simplicial sets and category theory to build the final global graph.

2.3 Mathematical Preliminaries: Tools for Topological Modeling

In the previous section, we arrived at a fundamental challenge: our attempt to respect the local geometry of the data manifold left us with a set of N incompatible, asymmetric “distances.” The “distance” from x_i to x_j is not the same as the “distance” from x_j to x_i . This means we cannot use the mathematics of standard metric spaces.

It translates the problem from the rigid world of metric geometry into the more flexible language of **topology**, specifically using tools from algebraic topology and category theory. This section serves as a self-contained primer on these key concepts. The recurring theme is one of “translation”—using formal mathematical structures to move a problem from a domain where it is difficult to solve (like “stitching” asymmetric metrics) into one where a solution becomes natural and elegant.

2.3.1 Simplicial Complexes and Simplicial Sets

Topology is the study of the “shape” of data—specifically, the properties of spaces that are preserved under continuous deformations like stretching, twisting, or bending (but not tearing or gluing). A primary tool for giving a “shape” to a “cloud” of data points is the **simplicial complex**.

Simplicial Complexes The basic building blocks of a simplicial complex are **n-simplices**, which are simple geometric objects corresponding to the convex hull of $n + 1$ independent points in Euclidean space.

- A **0-simplex** is a point (a vertex).
- A **1-simplex** is a line segment (an edge).

- A **2-simplex** is a filled-in triangle (a face).
- A **3-simplex** is a solid tetrahedron.

And so on. Each n -simplex has “faces” which are themselves simplices of lower dimension (e.g., the faces of a triangle are its three edges).

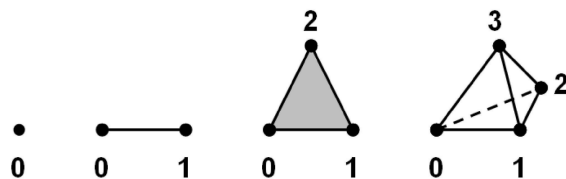


Figure 2.3: *Examples of n -simplices for $n=0, 1, 2, 3$. Geometric realizations of n -simplices for $0 \leq n \leq 3$.*

A **simplicial complex** is a collection of simplices together along their faces in a specific, consistent way. The rules are simple:

1. If a simplex is in the complex, all of its faces must also be in the complex. (e.g., if you include a triangle, you *must* also include its three edges and three vertices).
2. The intersection of any two simplices in the complex must be a single face shared by both. (e.g., two triangles can only be glued together along a single shared edge).

Figure 2.4 shows a simplicial complex built from points, edges, and triangles. This structure provides a discrete, combinatorial “skeleton” that approximates the shape of a continuous topological space.

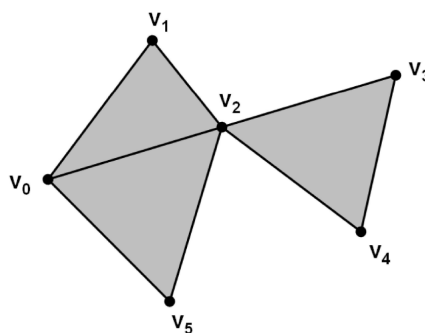


Figure 2.4: *A simplicial complex X . An example of a simplicial complex, constructed by gluing together points (0-simplices), edges (1-simplices), and triangles (2-simplices).*

Simplicial Sets: The Abstract Recipe While simplicial complexes are intuitive, their geometric definition is restrictive and computationally awkward. A more abstract and powerful generalization is the **simplicial set**.

Instead of thinking of a “geometric object,” think of a simplicial set as a *combinatorial recipe* for building one. It is defined not by geometry, but by abstract algebra and category theory. A simplicial set is a collection of sets X_n (the set of n -simplices) for $n \geq 0$, equipped with maps that describe the “gluing” rules:

- **Face maps** $d_i : X_n \rightarrow X_{n-1}$: These maps tell you how to find the faces of an n -simplex. For example, for a 1-simplex (edge) $e \in X_1$, $d_0(e)$ is its 0-th vertex (e.g., the “end point”) and $d_1(e)$ is its 1st vertex (e.g., the “start point”).
- **Degeneracy maps** $s_i : X_n \rightarrow X_{n+1}$: This is the clever part that simplicial complexes lack. These maps allow for “collapsed” or “degenerate” simplices. For example, $s_0(v)$ for a vertex $v \in X_0$ could create a “degenerate edge” in X_1 that just starts and ends at v . This flexibility is crucial for modeling complex topological structures and for the category-theoretic constructions UMAP relies on.

These maps must satisfy certain “compatibility relations” (which we don’t need to detail) that just ensure the “gluing” is consistent (e.g., the faces of a face of a simplex are also faces of the original simplex). The key takeaway is that a simplicial set is a more general and flexible “recipe for a shape” than a rigid simplicial complex.

2.3.2 A Primer on Category Theory: The Language of Translation

Why introduce this new layer of abstraction? Because it gives us access to **category theory**, a high-level language for describing mathematical structures and the relationships between them. It formalizes the concept of “translation” between different mathematical worlds, which is *exactly* what UMAP needs to do.

A **Category** $\mathcal{C}$ is a system of “things” and “maps” between them. It consists of:

- A collection of *objects*.
- A collection of *morphisms* (or arrows) between those objects.

These must satisfy three simple conditions: morphisms can be composed (if you have a map $f : A \rightarrow B$ and $g : B \rightarrow C$, you must have a map $g \circ f : A \rightarrow C$), composition is associative, and every object has an “do nothing” identity morphism.

Examples of Categories:

- **Set**: The category where objects are *sets* and morphisms are *functions*.
- **Vect**: The category where objects are *vector spaces* and morphisms are *linear maps*.
- **Top**: The category where objects are *topological spaces* and morphisms are *continuous maps*.

A **Functor** $F : \mathcal{C} \rightarrow \mathcal{D}$ is the “translator.” It is a map *between two categories*. A functor is a recipe that:

1. Maps every object X in $\mathcal{C}$ to an object $F(X)$ in $\mathcal{D}$.

2. Maps every morphism $f : X \rightarrow Y$ in $\mathcal{C}$ to a morphism $F(f) : F(X) \rightarrow F(Y)$ in $\mathcal{D}$.

This translation must be structure-preserving: it must preserve composition and identity morphisms. A functor is a principled way to “turn a vector space into a set” or “turn a set into a topological space.”

An **Adjunction** $F \dashv G$ is a special, profound relationship between two functors going in opposite directions, $F : \mathcal{C} \rightarrow \mathcal{D}$ and $G : \mathcal{D} \rightarrow \mathcal{C}$. It formalizes the idea that F and G are “almost” inverses, but in a weaker and more flexible way. A classic example is the adjunction between **Set** and **Vect**.

- Let $G : \mathbf{Set} \rightarrow \mathbf{Vect}$ be the “free” functor that takes a set S and constructs the vector space that has S as its basis. It *adds structure*.
- Let $F : \mathbf{Vect} \rightarrow \mathbf{Set}$ be the “forgetful” functor that takes a vector space V and just gives you its underlying set of points, *forgetting* the vector addition and scalar multiplication rules.

These are not inverses. But the adjunction $G \dashv F$ provides a formal, natural correspondence between maps $S \rightarrow F(V)$ (in **Set**) and maps $G(S) \rightarrow V$ (in **Vect**). It provides a principled translation between these two different worlds. This concept of “translation” is the key to UMAP’s theoretical solution.

2.3.3 Fuzzy Set Theory

The final piece of the puzzle is **fuzzy set theory**. Standard set theory is binary: an element is either *in* a set or it is *not* in the set. This is too rigid for real data.

A **fuzzy set** generalizes this by allowing for *graded membership*. A fuzzy set consists of a base set of objects A and a membership function $\mu : A \rightarrow [0, 1]$, where $\mu(a)$ represents the “degree of membership” of element a in the fuzzy set.

This is a perfect tool for our problem! We don’t know if x_i and x_j are “connected” or “not connected” (i.e., if the 1-simplex (x_i, x_j) is “in the set” or “not”). But we *do* have our asymmetric, locally-scaled values, which we can interpret as a *probability* or *confidence* of connection. We can say the edge (x_i, x_j) has a membership strength of, say, 0.8, while an edge (x_i, x_k) has a strength of 0.2.

Just like other mathematical systems, we can define the category **Fuzz**. Its objects are fuzzy sets, and its morphisms are functions $f : A \rightarrow B$ between the underlying sets that are non-decreasing with respect to membership strength (i.e., $\mu_B(f(a)) \geq \mu_A(a)$). This just means the maps “respect” the fuzzy membership.

2.3.4 Fuzzy Simplicial Sets

We can now combine these concepts to create the central mathematical object used by UMAP: the **fuzzy simplicial set**.

Let’s build this up intuitively:

1. A **simplicial set** is a “recipe” for a topological shape, made of sets of simplices $(X_0, X_1, X_2, \dots)$ and “gluing maps” (face/degeneracy maps).

2. A **fuzzy set** is a set where every element has a *membership strength* between 0 and 1.
3. A **fuzzy simplicial set** is simply a simplicial set where each set of simplices X_n is a *fuzzy set*.

This means that every simplex (every point, edge, triangle, etc.) is not just “present” or “absent,” but has a membership strength between 0 and 1. This structure is perfectly suited for UMAP’s goal:

- The **simplicial set** structure provides the combinatorial “skeleton” representing the data’s topology.
- The **fuzzy** membership values encode the *uncertainty* or *strength* of the connections. Our computed asymmetric “distance” from x_i to x_j can be directly translated into the *membership strength* of the 1-simplex (edge) (x_i, x_j) .

Formally, this is defined using the language of category theory. The “simplex category,” Δ , is a category whose objects are the “blueprints” for simplices $[n] = \{0, \dots, n\}$ and whose morphisms are the abstract “gluing” rules.

- A **simplicial set** is a functor $X : \Delta^{op} \rightarrow \mathbf{Set}$. This functor is a “recipe” that maps the blueprint $[n]$ to the *set* of n -simplices X_n .
- A **fuzzy simplicial set** is a functor $X : \Delta^{op} \rightarrow \mathbf{Fuzz}$. This is the *exact same idea*, but now the recipe maps the blueprint $[n]$ to the *fuzzy set* of n -simplices X_n .

This elegant categorical construction gives us a single mathematical object that can hold all of our messy, asymmetric, local “distance” information, turning it into a valid, globally-consistent topological structure.

2.4 Constructing the High-Dimensional Fuzzy Topological Representation

In the previous sections, we established the core challenge: by respecting the local geometry of the data manifold, we created N different, incompatible “views” of the data’s connectivity. The “distance” from $x_i \rightarrow x_j$ (using σ_i) was not equal to the “distance” $x_j \rightarrow x_i$ (using σ_j).

This section details how UMAP masterfully resolves this problem. It “translates” this collection of messy, asymmetric local distances into a *single, globally consistent topological structure*. This is the mathematical heart of the algorithm.

2.4.1 The Problem: A World of Asymmetric “Distances”

First, let’s be precise about what we created. For each data point x_i , we have a distance function d_{x_i} defined by our scaling factors σ_i . This function defines a “space” centered at x_i . This is not a standard metric space. It’s a more general object where distances can be asymmetric and even infinite (if two points are not in the local neighborhood).

The abstract mathematical category for these objects is **Finite Extended-Pseudo-Metric Spaces (FinEPMet)**. This is a fancy name, but it simply describes a “world” where our N local, asymmetric views “live.” We need to find a way to translate out of this world and into the world of topology.

2.4.2 The Solution: A Principled “Translation”

The core theoretical breakthrough of UMAP is the discovery of a formal, structure-preserving “translator” between this messy world of metric spaces and the clean, flexible world of topological spaces (specifically, fuzzy simplicial sets). This translator is known as an **adjunction**.

Theorem T. here is an **adjunction** between the category of finite extended-pseudo-metric spaces, **FinEPMet**, and the category of finite fuzzy simplicial sets, **Fin-sFuzz**.

An adjunction is a deep concept, but for our purposes, it provides a pair of “translation functions,” called **functors**, that move between these two mathematical worlds. We get:

- A functor $\text{FinReal} : \text{Fin-sFuzz} \rightarrow \text{FinEPMet}$ (Turns a fuzzy shape *into* a metric space).
- A functor $\text{FinSing} : \text{FinEPMet} \rightarrow \text{Fin-sFuzz}$ (Turns a metric space *into* a fuzzy shape).

We are interested in the second functor, **FinSing**, as it provides the explicit recipe for converting each of our N local metric spaces (D, d_{x_i}) into a corresponding local fuzzy simplicial set.

2.4.3 Step 1: Translate Local Distances into Local Probabilities

While the full definition of the **FinSing** functor is abstract, its practical consequence for 1-simplices (edges) is simple and elegant. It tells us *how* to convert a distance $d(x_j, x_k)$ into a fuzzy membership strength $\mu(\{x_j, x_k\})$. This relationship is given by:

$$\mu(\{x_j, x_k\}) = \exp(-d(x_j, x_k)) \quad (2.2)$$

This formula is not an arbitrary heuristic. It is a direct consequence of the adjunction theorem and provides the “correct” way to translate a distance (a value from 0 to ∞) into a membership strength (a value from 1 to 0).

- If $d(x_j, x_k) = 0$, then $\mu = \exp(0) = 1$ (a perfect connection).
- If $d(x_j, x_k) \rightarrow \infty$, then $\mu = \exp(-\infty) \rightarrow 0$ (no connection).

We apply this translation to *all* our local scaled distances $d_{x_i}(x_i, x_j) = \frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}$. This gives us N separate *local fuzzy simplicial sets*. Each set represents the topological structure from the “viewpoint” of a single data point, with membership strengths $\mu_i(x_i, x_j)$ representing the “probability” of the edge (x_i, x_j) existing from x_i ’s perspective.

2.4.4 Step 2: “Stitch” Local Views into a Global Whole

We now have N local “recipes,” but we need a single global one. The final step is to combine, or “stitch together,” all N local fuzzy sets into one. This is achieved by taking the **fuzzy set union** of all of them.

How do we combine the (asymmetric) membership strengths?

- From x_i ’s view, we have a strength $\mu_i(x_i, x_j)$.
- From x_j ’s view, we have a strength $\mu_j(x_j, x_i)$.

Since $\mu_i \neq \mu_j$, which one do we choose for the final, symmetric edge (x_i, x_j) ?

UMAP’s solution is to interpret these membership values as probabilities.

- Let $A = \mu_i(x_i, x_j)$ be the probability that the edge exists, according to x_i .
- Let $B = \mu_j(x_j, x_i)$ be the probability that the edge exists, according to x_j .

We want to find the probability that the edge exists according to *at least one* of them. This is the standard probability for the union of two events: $P(A \cup B) = P(A) + P(B) - P(A)P(B)$.

This specific formula for a fuzzy union is called the **probabilistic t-conorm**. The membership strength v_{ij} for the final, global edge (x_i, x_j) is:

$$v_{ij} = \mu_i(x_i, x_j) + \mu_j(x_j, x_i) - \mu_i(x_i, x_j) \cdot \mu_j(x_j, x_i) \quad (2.3)$$

This operation brilliantly resolves the asymmetry. The resulting value v_{ij} is perfectly symmetric ($v_{ij} = v_{ji}$) and represents a principled combination of both local viewpoints.

The result of this process is a single, large, weighted undirected graph. This graph is the 1-skeleton of our final fuzzy simplicial set, V . It is a single, unified topological representation of the high-dimensional data, capturing both local and global structure.

2.5 Optimization: Finding the Low-Dimensional Embedding

Having constructed our “target”—a fuzzy topological representation V of the high-dimensional data—the second phase of UMAP is to create the low-dimensional visualization.

The goal is to arrange a new set of points $Y = \{y_1, \dots, y_N\}$ in a low-dimensional space (e.g., $\mathbb{R}^2$) such that their *own* topological structure, let’s call it W , is as similar as possible to V .

This is an optimization problem: **Find the optimal coordinates Y that make W “look like” V .**

2.5.1 Step 1: Building the Low-Dimensional Fuzzy Set

This process mirrors the high-dimensional construction, but with one crucial difference: we are not *inferring* a manifold. We are *defining* the structure. The “manifold” is just the flat Euclidean space $\mathbb{R}^2$ (or $\mathbb{R}^m$).

We need a function that takes the simple Euclidean distance $\|y_i - y_j\|$ between two points in the low-D embedding and converts it into a membership strength w_{ij} . This function must:

1. Map distances $[0, \infty)$ to strengths $[1, 0]$.
2. Be differentiable, so we can use gradient-based optimization.
3. Be flexible, so we can control the “look” of the final embedding.

UMAP chooses a specific family of curves parameterized by a and b :

$$w_{ij} = \left(1 + a\|y_i - y_j\|_2^{2b}\right)^{-1} \quad (2.4)$$

The hyperparameters a and b are not set directly. Instead, they are fit from the user-friendly hyperparameter `min_dist`, which controls how “clumped” the final embedding should look. (Notably, setting $a = 1$ and $b = 1$ recovers the Student’s t-distribution used by t-SNE, making UMAP a generalization of this).

2.5.2 Step 2: Comparing the High-D and Low-D Structures

We now have two fuzzy sets (graphs) on the same N vertices:

- V : The **target** structure from high-D data, with fixed weights v_{ij} .
- W : The **current** structure from the low-D layout, with variable weights $w_{ij}(Y)$ that depend on the point locations Y .

We need a “cost function” that measures how different V and W are. UMAP uses the **cross-entropy** cost function, which is a standard way to measure the dissimilarity between two sets of probabilities (or fuzzy memberships):

$$C(V, W) = \sum_{i \neq j} \underbrace{\left[v_{ij} \log \left(\frac{v_{ij}}{w_{ij}} \right) \right]}_{\text{Attractive Term}} + \underbrace{\left[(1 - v_{ij}) \log \left(\frac{1 - v_{ij}}{1 - w_{ij}} \right) \right]}_{\text{Repulsive Term}} \quad (2.5)$$

The goal is to find the point locations Y that **minimize** this total cost C . This is done using an iterative optimization method, typically **Stochastic Gradient Descent (SGD)**.

2.5.3 Step 3: The “Force-Directed” Layout

The easiest way to understand this optimization is to think of it as a **physical simulation with forces**. The gradient of the cost function (which SGD uses to update the points) can be broken into two parts:

1. **Attractive Force:** The first term, $v_{ij} \log(\dots)$, is large when v_{ij} is large (i.e., x_i, x_j are neighbors in high-D). This term creates a force that **pulls** y_i and y_j **together** in the low-D space, trying to make w_{ij} large to match v_{ij} .
2. **Repulsive Force:** The second term, $(1 - v_{ij}) \log(\dots)$, is large when v_{ij} is small (i.e., x_i, x_j are *not* neighbors). This term creates a force that **pushes** y_i and y_j **apart**, trying to make w_{ij} small to match v_{ij} .

UMAP’s Secret Weapon for Global Structure The key to UMAP’s excellent preservation of global structure lies in that repulsive force. The term $(1 - v_{ij})$ acts as the “weight” for the repulsion.

- For pairs that are very far apart in high-D, $v_{ij} \approx 0$.
- This makes the repulsive weight $(1 - v_{ij}) \approx 1$.

This means that even widely separated points feel a *strong, meaningful repulsive force*. This is what keeps large, disconnected clusters from mixing together in the final plot. This is a major advantage over t-SNE, whose repulsive force “fades out” quickly, causing it to struggle with global structure.

UMAP’s Secret Weapon for Scalability There is one final problem: calculating the total repulsion involves a sum over *all pairs* of points, an $O(N^2)$ operation. For $N = 1,000,000$, this is 10^{12} calculations per step, which is intractable.

UMAP solves this with **negative sampling**. Instead of calculating the repulsion for all N^2 pairs, the algorithm does the following:

1. For each “positive sample” (an edge (i, j) with $v_{ij} > 0$), calculate the **attractive force**.
2. For that same edge, randomly select a small number of “negative samples”—points k for which the edge (i, k) does *not* exist (or has $v_{ik} \approx 0$).
3. **Only** calculate the **repulsive force** for these few negative samples.

This provides a very fast, efficient stochastic approximation of the true gradient. It’s this combination of cross-entropy with negative sampling that allows UMAP to be both more accurate at preserving global structure *and* significantly faster and more scalable than previous methods.

2.6 From Theory to Practice: The UMAP Algorithm

The theoretical framework described so far is elegant, but its success hinges on two practical challenges:

1. **Initialization:** Our low-dimensional embedding Y is found by minimizing a complex cost function. Where in the low-dimensional space do we place our points $\{y_i\}$ *before* we start the optimization?
2. **Tuning:** How do we, as users, control the algorithm’s behavior to suit our data and our questions?

This section details the practical, computational steps that answer these questions, bridging the abstract concepts with their concrete implementation.

2.6.1 The Initialization Problem: Avoiding Bad Starts

The optimization process in UMAP is an iterative one (Stochastic Gradient Descent, or SGD). Think of this as dropping a ball on a vast, bumpy landscape (the “cost function”) and letting it roll downhill to find the lowest point (the “optimal embedding”).

A **random initialization**, as used in many t-SNE implementations, is like dropping the ball at a random location. In a complex landscape, this is risky. The ball will roll to the *nearest* valley, but this small local minimum (a small dip) is almost certainly not the *global minimum* (the deepest valley in the entire landscape). This can lead to slow convergence and embeddings that are “stuck” in a poor configuration, misrepresenting the data’s global structure.

UMAP employs a much more sophisticated strategy: **spectral initialization**. The goal is to get a “head start”—to place the points in an initial configuration that already reflects the coarse, “big picture” global topology of the data. The subsequent SGD optimization then merely needs to refine this layout, arranging the fine-grained local structures correctly.

2.6.2 Spectral Initialization: Using the Graph’s “Vibration”

This method leverages the high-dimensional fuzzy topological representation (our graph V) that we worked so hard to build. This graph’s structure contains all the information we have about the manifold.

1. **From Graph to Matrix:** We first represent our fuzzy graph V as a large $N \times N$ **weighted adjacency matrix**, A , where the entry $A_{ij} = v_{ij}$ (the final, symmetrized membership strength from Section 3).
2. **Constructing the Graph Laplacian:** From this adjacency matrix, we construct a new matrix called the **graph Laplacian**, L . This is a central object in spectral graph theory.
 - First, create a **degree matrix** D . This is a diagonal matrix where each diagonal entry D_{ii} is the sum of all connection strengths for that point: $D_{ii} = \sum_j A_{ij}$.
 - The (unnormalized) graph Laplacian is then defined as $L = D - A$.
3. **The Intuition Behind the Laplacian:** What is this L matrix? It’s a “difference operator” that describes the relationships between points and their neighbors. Think of it as describing a network of masses (points) and springs (edges). The Laplacian matrix governs how “vibrations” or forces would propagate through this system.
4. **The Deeper Connection:** The reason this is so powerful is that the **graph Laplacian** L is a discrete, computational approximation of the **Laplace-Beltrami operator** $\Delta_{\mathcal{M}}$ from continuous geometry. This operator is a fundamental tool for understanding the geometry of manifolds. Its “natural vibrations” (its eigenfunctions) reveal the manifold’s most essential, large-scale geometric properties.
5. **Using the Eigenvectors:** By analyzing our graph Laplacian L , we are *approximating* the fundamental geometry of the (unknown) manifold $\mathcal{M}$.

- We find the **eigenvectors** of L . These are the “principal components” or “natural vibration modes” of the graph.
 - The eigenvector for the smallest eigenvalue ($\lambda_0 = 0$) is trivial (it’s a constant vector) and gives no information.
 - The eigenvectors corresponding to the **smallest non-zero eigenvalues** ($\lambda_1, \lambda_2, \dots$) represent the “slowest” vibrations, which correspond to the *largest-scale global structures* of the graph. These are known as **Laplacian Eigenmaps**.
6. **The “Head Start”:** UMAP’s initialization procedure is to simply compute the eigenvectors for the m smallest non-zero eigenvalues of L (where m is our target dimension, e.g., $m = 2$). These two eigenvectors give us an x and y coordinate for each data point.

This initial layout Y_{init} is not random; it is a “best guess” of the global structure derived from the data’s own connectivity graph. This is a primary reason UMAP is so effective at preserving global structure. This procedure is detailed in Algorithm 4 of the UMAP paper.

2.6.3 Tuning the Algorithm: Key Hyperparameters

The behavior of this entire process is controlled primarily by two user-facing hyperparameters: `n_neighbors` and `min_dist`. Understanding them is key to using UMAP effectively.

- **n_neighbors:** This is arguably the most important hyperparameter. It directly controls the construction of the high-dimensional fuzzy graph V .
 - **How it works:** This parameter is the k in the normalization equation from Section 2: $\sum_{j=1}^k \exp(\dots) = \log_2(k)$. It defines the size of the local neighborhood that the algorithm “cares” about when learning the manifold’s structure.
 - **Small values** (e.g., 2 to 10): A small k means $\log_2(k)$ is small. The equation is satisfied by very small σ_i values, which creates a “steep” exponential decay. The algorithm will **focus intensely on the very local structure**, connecting only the absolute nearest neighbors.
 - * *Result:* The embedding will be highly detailed, preserving fine-grained clusters. It’s good for finding micro-clusters.
 - * *Risk:* It may fail to “see” the “big picture” connections, potentially breaking the global structure into many disconnected islands.
 - **Large values** (e.g., 50 to 200): A large k means $\log_2(k)$ is large. This requires larger σ_i values, creating a “gradual” exponential decay. The algorithm **averages its view over a much wider neighborhood**.
 - * *Result:* The embedding will better reflect the global “big picture” topology, connecting larger-scale structures.
 - * *Risk:* It will “smooth over” and potentially lose the fine-grained local details.
- **min_dist:** This parameter controls the “look” of the final plot by adjusting the cost function for the *low-dimensional* embedding.

- **How it works:** This parameter is used to fit the a and b parameters of the low-dimensional similarity curve, $w_{ij} = (1 + a\|y_i - y_j\|_2^{2b})^{-1}$. It does *not* change the high-dimensional graph V ; it only changes the “target” shape of the embedding Y .
- **Small values** (e.g., 0.0 to 0.1): This tells the algorithm to fit a curve that allows $w_{ij} \approx 1$ even for very small distances.
 - * *Result:* Points are allowed to be packed very tightly. This creates **dense, compact clusters**. This may be a more “honest” representation of the data’s topology but can suffer from overplotting in visualizations.
- **Large values** (e.g., 0.5 to 0.99): This tells the algorithm to fit a curve where w_{ij} starts to drop from 1 even at larger distances.
 - * *Result:* Points are **forced to be more spread out**. This creates visually appealing, “fluffier” embeddings that are less crowded, but at the cost of “lying” slightly about the true topological density.

The interplay of these parameters allows the user to tune the UMAP embedding for their specific goals, from exploratory visualization (favoring global structure and visual appeal) to feature extraction for machine learning (favoring a more faithful local/global balance). The effects of varying these parameters are illustrated in Figures 1, 2, and 3 of the original UMAP paper, which show how different settings can reveal different aspects of datasets like MNIST and PenDigits.

2.7 Empirical Evaluation and Conclusion

The theoretical elegance of UMAP’s mathematical framework is ultimately validated by its practical efficacy on real-world data. This section provides a qualitative and quantitative evaluation of UMAP’s performance in comparison to other standard dimensionality reduction techniques.

2.7.1 Qualitative Evaluation

Visual inspection of embeddings is a primary method for evaluating dimensionality reduction algorithms. As shown previously in Figure 2.1, UMAP consistently produces high-quality visualizations across a range of datasets. On the MNIST handwritten digits dataset, UMAP not only separates the ten digit classes into distinct clusters, similar to t-SNE, but it also arranges these clusters in a way that reflects their global relationships. For instance, the clusters for ‘4’, ‘7’, and ‘9’ are positioned near each other, as are ‘3’, ‘5’, and ‘8’, reflecting the visual similarity of these digits. Similarly, on the COIL20 object image dataset, which contains loops corresponding to object rotations, UMAP is more successful at preserving these loops and even their intertwining, a complex topological feature that other methods often fail to capture. This superior preservation of global structure is a consistent theme in qualitative comparisons and is a direct result of the algorithm’s design, from its topological foundations to its spectral initialization.

2.7.2 Quantitative Evaluation

To move beyond visual assessment, the quality of an embedding can be quantified by measuring how well it preserves the neighborhood structure of the original data. A common metric is the accuracy of a k-nearest neighbor (kNN) classifier trained on the low-dimensional representation. This provides an objective measure of how well the local structure, which is critical for classification, has been maintained.

By varying the value of k in the kNN classifier, one can probe the preservation of structure at different scales. A small k tests the preservation of very local neighborhoods, while a large k tests the preservation of more non-local, or global, structure. Table 2.1 summarizes the performance of UMAP against other algorithms on the COIL-20 and PenDigits datasets.

Table 2.1: kNN Classifier accuracy (mean $\pm$ std. dev.) for varying values of k over the embedding spaces of COIL-20 and PenDigits datasets. Data adapted from McInnes et al. (2018).

Algorithm	k=10	k=40	k=160
COIL-20 Dataset			
UMAP	0.921 (± 0.075)	0.904 (± 0.056)	0.803 (± 0.138)
t-SNE	0.934 (± 0.115)	0.857 (± 0.125)	0.609 (± 0.067)
LargeVis	0.888 (± 0.092)	0.833 (± 0.106)	0.616 (± 0.066)
Laplacian Eigenmaps	0.629 (± 0.153)	0.578 (± 0.159)	0.446 (± 0.110)
PCA	0.667 (± 0.179)	0.620 (± 0.230)	0.375 (± 0.111)
PenDigits Dataset			
UMAP	0.973 (± 0.044)	0.954 (± 0.060)	0.951 (± 0.085)
t-SNE	0.977 (± 0.033)	0.956 (± 0.064)	0.949 (± 0.065)
LargeVis	0.966 (± 0.053)	0.959 (± 0.066)	0.921 (± 0.085)
Laplacian Eigenmaps	0.778 (± 0.113)	0.778 (± 0.112)	0.747 (± 0.108)
PCA	0.622 (± 0.092)	0.636 (± 0.078)	0.629 (± 0.107)

The results show that for very local structure (small k), UMAP’s performance is comparable to that of t-SNE and LargeVis. However, as k increases, testing the preservation of more global structure, UMAP’s performance remains high while that of t-SNE and LargeVis degrades significantly. On the COIL-20 dataset with $k = 160$, UMAP’s accuracy is substantially higher than its competitors, providing strong quantitative evidence for its superior global structure preservation.

Furthermore, UMAP demonstrates greater stability in its embeddings. Due to its stochastic nature, different runs of UMAP will produce slightly different results. An analysis using Procrustes distance to measure the similarity between embeddings of a full dataset and embeddings of subsamples showed that UMAP embeddings are significantly more stable and consistent under subsampling than those of t-SNE.

2.7.3 Conclusion

Uniform Manifold Approximation and Projection is more than an incremental improvement upon existing dimensionality reduction techniques; it is the practical realization of a

deep and cohesive mathematical theory. By grounding itself in the principles of Riemannian geometry and algebraic topology, UMAP provides a powerful framework for modeling the structure of high-dimensional data. Its core components—the assumption of a locally constant Riemannian metric, the construction of a fuzzy simplicial set representation via a formal category-theoretic adjunction, and the optimization of a cross-entropy cost function—are not ad-hoc heuristics but are logically connected elements derived from a unified set of first principles.

This strong theoretical foundation translates directly into tangible benefits: high-quality embeddings that balance the preservation of local and global structure, superior computational performance and scalability, and enhanced stability. By successfully navigating the trade-offs that have long defined the field of dimensionality reduction, UMAP has established itself as a versatile and indispensable tool for data scientists and researchers across a multitude of disciplines.

Chapter 3

kPCA

3.1 PCA

Principal Component Analysis (PCA) is a well-known linear dimensionality reduction technique widely used for data visualization and feature extraction. It transforms the data into a new coordinate system such that the directions (called principal components) correspond to the axes of maximum variance in the data. Each principal component is a linear combination of the original features, representing the directions along which the data varies the most. By retaining only the top few principal components that capture the majority of the total variance, PCA effectively reduces the dimensionality of the dataset while preserving most of its essential structure and information.

3.1.1 Data and notation

Let $X \in \mathbb{R}^{n \times p}$ denote the data matrix whose i -th row is the transpose of the observation $X_i^\top \in \mathbb{R}^{1 \times p}$, where $X_1, \dots, X_n$ are iid samples of a random vector $X \in \mathbb{R}^p$. The sample mean is

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \in \mathbb{R}^p,$$

and the centered data matrix is

$$\tilde{X} = X - \mathbf{1}_n \bar{X}^\top,$$

where $\mathbf{1}_n$ is the n -vector of ones. The sample (unbiased) covariance matrix is

$$S = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})^\top = \frac{1}{n-1} \tilde{X}^\top \tilde{X} \in \mathbb{R}^{p \times p}.$$

Often it is useful to standardize each variable to zero mean and unit variance before performing PCA. Let

$$s_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_{ij} - \bar{X}_j)^2} \quad (j = 1, \dots, p)$$

be the sample standard deviations, and let $D = \text{diag}(s_1, \dots, s_p)$. The standardized (z-scored) centered data matrix is

$$Z = \tilde{X}D^{-1},$$

and PCA performed on Z is equivalent to applying PCA to the sample correlation matrix R instead of S .

In this report we will assume data are mean-centered. If variables are measured on very different scales or in different units, we will standardize (z-score) them and use the correlation matrix; otherwise we will use the covariance matrix S .

WLOG we assume S is full-rank.

3.1.2 PCA formulation and results

Formulation for the first principal component Find a transformation $Y_1 = w_1^\top X$ (Y_1 will be the first principal component) on X s.t. w_1 maximizes $w_1^\top S w_1$ subject to $\|w_1\| = 1$. The Euler–Lagrange condition gives $S w_1 = \lambda_1 w_1$, so the maximizer is the eigenvector of S with largest eigenvalue.

Formulation for the subsequent principal components $Y_2, \dots, Y_p$ For the subsequent principal components, we find a transformation $Y_i = w_i^\top X$ (Y_i will be the i^{th} principal component) on X s.t. w_i maximizes $w_i^\top S w_i$ subject to $\|w_i\| = 1$ and $\langle w_i, w_j \rangle = 0 \ \forall j < i$. The Euler–Lagrange condition gives $S w_i = \lambda_i w_i$, so the maximizer is the eigenvector of S with largest eigenvalue.

Result Let S be the var-cov matrix of X . Let (λ_i, e_i) be the i^{th} eigen value-eigen vector pair for S for $i = 1, \dots, p$ where e_i 's are orthonormal.

WLOG let $\lambda_1 \geq \lambda_2 \dots \geq \lambda_p$. Then the i^{th} principal component is

$$Y_i = e_i^\top = \sum_{k=1}^p e_{ik} X_k$$

and

$$\text{Var}(Y_i) = e_i^\top S e_i = \lambda_i$$

and

$$\text{Cov}(Y_i, Y_j) = e_i^\top S e_j = 0 \text{ when } i \neq j$$

The above result follows from the following theorem.

Theorem 1. Consider any positive definite matrix $A_{p \times p}$

Let the eigen values of A are $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$. Let the normalised eigen-vectors be $e_1, e_2, \dots, e_p$.

Then $\forall x_{p \times 1}, x \neq 0$ we have

$$\max_{x \neq 0} \frac{x^\top A x}{x^\top x} = \lambda_1$$

and

$$\min_{x \neq 0} \frac{x^\top A x}{x^\top x} = \lambda_p$$

with equality attained for $x = e_1$ and e_p , respectively.

Moreover, $\forall x \neq 0$ and $x \perp e_1, e_2, \dots, e_k$ the following holds

$$\max_{x \neq 0, \perp e_1, e_2, \dots, e_k} \frac{x^\top A x}{x^\top x} = \lambda_{k+1}$$

with equality attained at $x = e_{k+1}$

Proof: Let the spectral decomposition of A be

$$A = \sum_{i=1}^p \lambda_i e_i e_i^\top$$

Construct an orthogonal matrix $P = [e_1 e_2 \dots e_p]$ and let $L = \text{diag}(\lambda_1 \lambda_2 \dots \lambda_p)$

Therefore A can be written as $A = P L P^\top$

Hence we find $A^{-1} = P L^{-1} P^\top$ and $A^{1/2} = P L^{1/2} P^\top$

let $y = P^\top x$

$$\frac{x^\top A x}{x^\top x} = \frac{x^\top A^{1/2} A^{1/2} x}{x^\top P P^\top x} = \frac{y^\top L y}{y^\top y}$$

$$= \frac{\sum_{i=1}^p \lambda_i y_i^2}{\sum_{i=1}^p y_i^2} \leq \lambda_1$$

equality is attained at $x = e_1$

similarly we get the minimum value is λ_p which is attained at $x = e_p$.

Finally, we have $y = P^\top x$ i.e. $x = P y = \sum_{i=1}^p e_i y_i$

assume that $x \perp e_1, e_2, \dots, e_k$

Therefore, $x^\top e_j = 0$ for $j = 1, \dots, k$

$$\text{also, } e_j^\top x = e_j^\top \sum_{i=1}^p e_i y_i = \sum_{i=1}^p e_j^\top e_i y_i = y_j$$

Therefore, $y_j = 0$ for $j = 1, \dots, k$

$$\text{Hence, } \frac{x^\top A x}{x^\top x} = \frac{y^\top L y}{y^\top y} = \frac{\sum_{i=k+1}^p \lambda_i y_i^2}{\sum_{i=k+1}^p y_i^2} \leq \lambda_{k+1}$$

Hence, maximum value is λ_{k+1} and is attained when $y_{k+1} = 1$ and rest are 0 i.e. $x = e_{k+1}$.

How to do PCA? In order to do PCA, construct the S matrix from the data, solve the eigen value problem and obtain the eigen values and eigen vectors. Take the top-k eigen values which describe the desired level of variance in the data.

3.1.3 Why kPCA?

Although Principal Component Analysis (PCA) is one of the most widely used techniques for dimensionality reduction, it is fundamentally a **linear** method. PCA assumes that the directions of maximum variance in the data can be captured through linear combinations of the original features. However, many real-world datasets exhibit **nonlinear** structures, where the data lie on or near a curved manifold in the high-dimensional feature space. In such cases, a linear projection obtained through PCA fails to uncover the intrinsic geometry of the data.

Kernel Principal Component Analysis (kPCA) addresses this limitation by **mapping the input data into a high-dimensional feature space** through a nonlinear mapping $\Phi(x)$, and then performing standard PCA in that space. Instead of explicitly computing the mapping, kPCA uses the **kernel trick** to evaluate inner products in the feature space using a kernel function $k(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$. This allows PCA to capture nonlinear relationships between data points while retaining the computational framework of linear algebra.

As a result, kPCA can extract nonlinear principal components that better represent the underlying data distribution. It is particularly effective for problems involving complex or manifold-structured data such as image recognition, denoising, novelty detection, and nonlinear feature extraction. In summary, kPCA extends the power of PCA to a broader class of data by combining the strengths of kernel methods and eigenvalue-based dimensionality reduction.

3.1.4 Kernels

A *kernel* provides a way to work with nonlinear feature representations implicitly as discussed in [5]. Formally, let

$$\Phi : \mathcal{X} \rightarrow \mathcal{F}$$

be a mapping from the input space $\mathcal{X} \subseteq \mathbb{R}^p$ into a (typically higher- or infinite-dimensional) feature space $\mathcal{F}$ equipped with an inner product $\langle \cdot, \cdot \rangle_{\mathcal{F}}$. For two inputs $x, y \in \mathcal{X}$ we write

$$\langle \Phi(x), \Phi(y) \rangle_{\mathcal{F}}$$

for their inner product in feature space. In many algorithms it is enough to know these inner products rather than the explicit coordinates of $\Phi(x)$. A *kernel function* $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is defined by

$$k(x, y) = \langle \Phi(x), \Phi(y) \rangle_{\mathcal{F}}.$$

This identity is the basis of the *kernel trick*: any algorithm that can be written only in terms of inner products can be executed in $\mathcal{F}$ by replacing inner products with $k(\cdot, \cdot)$, avoiding explicit computation of $\Phi(x)$.

The kernel matrix. Given a sample $x_1, \dots, x_n$, the *kernel matrix* $K \in \mathbb{R}^{n \times n}$ is

$$K_{ij} = k(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle_{\mathcal{F}}.$$

Important properties of K :

- K is symmetric: $K^\top = K$.
- K is positive semidefinite (PSD): for any $\alpha \in \mathbb{R}^n$,

$$\alpha^\top K \alpha = \left\langle \sum_{i=1}^n \alpha_i \Phi(x_i), \sum_{j=1}^n \alpha_j \Phi(x_j) \right\rangle_{\mathcal{F}} = \left\| \sum_{i=1}^n \alpha_i \Phi(x_i) \right\|_{\mathcal{F}}^2 \geq 0.$$

- Eigen-decomposition of K reveals directions in feature space spanned by the training images $\Phi(x_i)$; this is the reason K is central to kernel PCA and other spectral kernel methods.

Centering the kernel matrix. PCA requires centered data. If the feature vectors $\Phi(x_i)$ are not explicitly available, the centered kernel matrix can be computed directly. Let

$$H = I_n - \frac{1}{n} \mathbf{1} \mathbf{1}^\top$$

be the centering matrix (here $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector). The centered kernel matrix is

$$K_c = HKH.$$

This formula is used in kPCA for centering in $\mathcal{F}$ without forming $\Phi(x)$.

Common kernel functions.

- **Linear kernel:** $k(x, y) = x^\top y$. This corresponds to Φ being the identity map.
- **Polynomial kernel:** $k(x, y) = (x^\top y + c)^d$. This has a finite-dimensional Φ consisting of all monomials up to degree d .
- **Gaussian / RBF kernel:** $k(x, y) = \exp(-\|x - y\|^2 / (2\sigma^2))$. The corresponding Φ is infinite-dimensional; the kernel nevertheless yields a valid PSD kernel matrix and is very flexible for capturing local, nonlinear structures.

Mercer's theorem Let k be a kernel function and $x_1, \dots, x_n$ be a set of samples. Then k is a valid kernel function iff

- K is symmetric
- K is positive semi-definite

3.2 kPCA

In this section we will see how we can use kernel-trick with PCA to get a new method for non-linear dimensionality reduction as described in [5].

3.2.1 Data and Notation

Let $X \in \mathbb{R}^{n \times p}$ denote the data matrix whose i -th row is the transpose of the observation $X_i^\top \in \mathbb{R}^{1 \times p}$, where $X_1, \dots, X_n$ are iid samples of a random vector $X \in \mathbb{R}^p$. Let Φ be the mapping from $\mathbb{R}^p$ to some space $\mathcal{F}$. The sample mean is

$$\Phi(\bar{X}) = \frac{1}{n} \sum_{i=1}^n \Phi(X_i) \in \mathcal{F},$$

and the centered data matrix (for both finite and infinite dimension) is

$$\tilde{\Phi}(X) = \Phi(X) - \mathbf{1}_n \Phi(\bar{X})^\top,$$

where $\mathbf{1}_n$ is the n -vector of ones. The kernel matrix is

$$\tilde{K} = HKH.$$

where

$$H = I_n - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$$

The S matrix here will be

$$S = \frac{1}{n} \sum_{j=1}^n \tilde{\Phi}(x_j) \tilde{\Phi}(x_j)^\top$$

3.2.2 kPCA Algorithm

Similar to PCA, we need to find eigen values and eigen vectors satisfying

$$SV = \lambda V, V \in \mathcal{F}$$

we get the system of equations

$$\tilde{\Phi}(x_k) \cdot SV = \lambda(\tilde{\Phi}(x_k) \cdot V) \text{ for all } k = 1, \dots, n$$

there exist coefficients s.t.

$$V = \sum_{i=1}^n \alpha_i \tilde{\Phi}(x_i)$$

we arrive at

$$n\lambda\tilde{K}\alpha = \tilde{K}^2\alpha$$

To find the solution, we solve the eigen value problem

$$n\lambda\alpha = \tilde{K}\alpha$$

Any additional solutions obtained from the above equation don't make a difference in the expansion of $V = \sum_{i=1}^n \alpha_i \tilde{\Phi}(x_i)$ so we don't need to worry about that.

We normalize the solutions α^k obtained from the above equation corresponding to non-zero eigen values s.t. $V^k \cdot V^k = 1$. From the above equations this can be equivalently written as

$$1 = \lambda_k(\alpha^k \cdot \alpha^k)$$

For any new vector $x \in \mathbb{R}^p$ we can compute the projections as follows

$$V^k \cdot \Phi(x) = \sum_{i=1}^n \alpha_i^k (\Phi(x_i) \cdot \Phi(x)) \text{ for all } k \text{ s.t. } \lambda_k > 0$$

For dimensionality reduction, we can take the top-k λ values which describe the desired level of variance and store the corresponding α^k 's for future calculations of the projections.

Chapter 4

SNE

In this report, we shall cover the dimensionality reduction method SNE, and its various forms. [1] [2]

SNE is a manifold learning and dimensionality reduction method with a probabilistic approach. SNE considers every point to be a neighbor of every other point in space with some probability, we try to preserve this probability in our reduced embedding space.

4.1 SNE

SNE considers gaussian distribution in both of the input and embedding space.

Consider a point x_i in our input space, we would consider a gaussian distribution around the point where x_i is at the mean and the distribution is for probability of accepting other points as it's neighbor. The variable of interest becomes the distance $d \in \mathbb{R}$ and Gaussian Probability is

$$f(d) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{d^2}{2\sigma^2}\right)$$

where mean of the distribution is assumed to be zero.

The constant factor $\frac{1}{\sqrt{2\pi}\sigma}$ could be dropped, but the resulting factor $\exp(-\frac{d^2}{2\sigma^2})$ does not add (integrate) to 1, hence could not be considered as a probability distribution. The problem is solved by dividing it by sum of all possible values of $\exp(-\frac{d^2}{2\sigma^2})$, resulting in the softmax function.

Therefore, the probability that the point $x_i \in \mathbb{R}^n$ is a neighbor of $x_j \in \mathbb{R}^n$ is

$$\mathbb{R} \ni p_{ij} = \frac{\exp(-d_{ij}^2)}{\sum_{k \neq i} \exp(-d_{ik}^2)}$$

where

$$\mathbb{R} \ni d_{ij}^2 = \frac{\|x_i - x_j\|_2^2}{2\sigma_i^2}$$

This method is also used in calculating the probabilities in reduced space and in general is widely used.

The variance σ_i^2 is the variance we consider for the probability distribution of x_i , it could be set to a fixed or number or determined by binary search to make the entropy of distribution a desired value.

Moving to the embedding space, we consider a gaussian probability around a point $y_i \in \mathbb{R}^d$ where d is the dimension of embedding space.

The probability that the point $y_i \in \mathbb{R}^d$ is a neighbor of $y_j \in \mathbb{R}^d$ is

$$\mathbb{R} \ni q_{ij} = \frac{\exp(-z_{ij}^2)}{\sum_{k \neq i} \exp(-z_{ik}^2)}$$

where

$$\mathbb{R} \ni z_{ij}^2 = ||y_i - y_j||_2^2$$

Here the variance in the embedding space is not used, (or set to 0.5), as variance in the embedding space is the choice of algorithm.

As the goal of the SNE is to preserve these probabilities, therefore the cost function to be minimized is chosen to be the sum of Kullback-Leibler Divergences over these n points.

$$\mathbb{R} \ni C = KL(P||Q) = \sum_{i=1}^n \sum_{j=1, j \neq i}^n p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

To solve this, we consider the partial derivatives with respect to the embedding space vector.

$$\mathbb{R}^d \ni \frac{\partial C}{\partial y_i} = 2 \sum_{j=1}^n (p_{ij} - q_{ij} + p_{ji} - q_{ji})(y_i - y_j),$$

where p_{ii} and q_{ii} are 0.

Gradient descent method is used to find out the embedding vector.

4.2 Symmetric SNE

We consider another definition for p_{ij} ,

$$\mathbb{R} \ni p_{ij} = \frac{\exp(-d_{ij}^2)}{\sum_{k \neq l} \exp(-d_{lk}^2)}$$

where

$$\mathbb{R} \ni d_{ij}^2 = \frac{||x_i - x_j||_2^2}{2\sigma_i^2}$$

Consider the following scenario, if some of the points x_i are an outlier in the dataset, the corresponding p_{ij} will be extremely small, because the denominator is fixed but the numerator will be small for outliers.

The issue in SNE version of the equation is that the denominator for all the points is not fixed, and therefore the denominator for an outlier will also be small waving out the problem of small numerator.

To over come this, we use:

$$\mathbb{R} \ni p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

where

$$\mathbb{R} \ni p_{j|i} := \frac{\exp(-d_{ij}^2)}{\sum_{k \neq i} \exp(-d_{ik}^2)}$$

here we have got ourselves a symmetric version of SNE.

The problem remains the same, minimizing the KL-Divergence.

The partial derivative of cost function now becomes,

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)$$

4.3 t-SNE

t-distributed Stochastic Neighbor Embedding (t-SNE) is a machine learning algorithm used for dimensional reduction and visualization of high-dimensional data.

Its main objective is the mapping of a high-dimensional space to a low-dimensional space (usually 2D or 3D) while preserving the dataset properties. The algorithm minimizes the Kullback-Leibler divergence between two probability distributions defined in the two spaces, in which the pairwise similarities are respected.

t-SNE solves a peculiar problem of crowding, which happens when we try to fit information into a lower dimension while trying to preserve the information about all points.

Instead of using the gaussian distribution for the embedding space as seen in t-SNE, we would be using Student-t distribution, as t-distribution has larger tails than gaussian distribution.

Consider points $x_i \in \mathbb{R}^n$ in the data space $y_i \in \mathbb{R}^m$ ($m < n$) in the corresponding reduced space, the pairwise similarity is defined by weighted sum of two conditional probabilities,

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

where

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2/2\sigma^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2/2\sigma^2)}$$

for our high dimensional space and

$$q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||y_k - y_l||^2)^{-1}}$$

for our low dimension space.

The dimensionality reduction strategy consists of minimizing the sum of Kullback-Leibler Divergence between conditional probabilities p_{ij} and q_{ij}

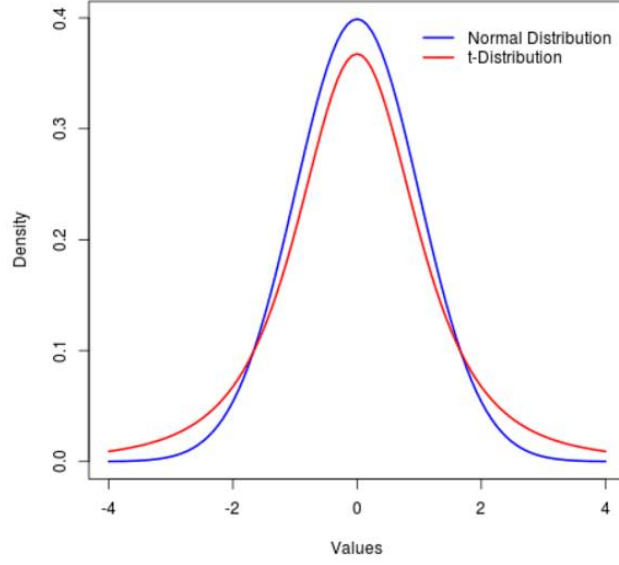


Figure 4.1: Gaussian and t-Distribution curve

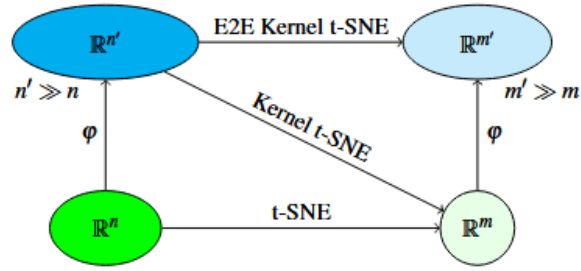


Figure 4.2: Dimensional reduction transformations

$$C = KL(P||Q) = \sum_i \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

The problem is solved by computing the partial divergence of the objective function with respect to the low dimension vector

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ji} - q_{ji})(1 + ||y_i - y_j||)^2 (y_i - y_j)$$

4.4 kernel t-SNE

In this method the reduction of the high dimensional data space to a low dimension embedding space is not done directly, instead the high dimension space is uplifted to a bigger space using non linear kernel functions and then the reduction to embedding space is done.

Method \ Dataset	t-SNE	kernel t-SNE	e2e kernel t-SNE
Cardio	90.78%	90.82%	92.81%
COIL-20	94.93%	85.16%	87.28%
MNIST	88.84%	88.89%	90.04%
Olivetti	91.13%	90.45%	91.92%
USPS	92.44%	92.45%	93.31%

Figure 4.3: Comparison of various kernel methods.

The non-linear map changes the metric and allows for the possibility of capturing of more complex patterns.

We redefine the conditional probability from t-SNE by mapping the points x_i to a lifted space $\varphi(x_i)$ using the function $\varphi(\cdot)$:

$$p_{j|i} = \frac{\exp(-\|\varphi(x_i) - \varphi(x_j)\|^2/2\sigma^2)}{\sum_{k \neq i} \exp(-\|\varphi(x_i) - \varphi(x_k)\|^2/2\sigma^2)}$$

The original method is adapted for this task by using the kernel trick for calculating the norm:

$$\|\varphi(x_i) - \varphi(x_j)\|^2 = \varphi(x_i)^T \varphi(x_i) - 2\varphi(x_i)^T \varphi(x_j) + \varphi(x_j)^T \varphi(x_j)$$

which leads to

$$\|\varphi(x_i) - \varphi(x_j)\|^2 = k(x_i, x_i) - 2k(x_i, x_j) + k(x_j, x_j),$$

where $k(u, v) = \varphi(u)^T \varphi(v)$ is the proposed kernel function.

Another major development is, also lifting the embedding space , and then minimizing the KL-Divergence.

Here, the conditional-probability for the embedding space becomes,

$$q_{ij} = \frac{(1 + \|\varphi(y_i) - \varphi(y_j)\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\varphi(y_k) - \varphi(y_l)\|^2)^{-1}}$$

The calculation for the partial dervative leads us to:

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{ij} - q_{ij})(1 + d_{ij})^{-1} \cdot \frac{\partial [k(y_i, y_i) - 2k(y_i, y_j)]}{\partial y_i}$$

, where we use the kernel trick in the reduced space,

$$d_{ij} = \|\varphi(y_i) - \varphi(y_j)\|^2 = k(y_i, y_i) - 2k(y_i, y_j) + k(y_j, y_j),$$

A common choice in practice is to use the Radial Basis Function kernel $k(x, y) = \exp(-\gamma\|x - y\|^2)$.

The results of the various SNE methods on some popular datasets are as follows, signifying that using the kernel methods did improve the results.

Bibliography

- [1] Benyamin Ghojogh, Ali Ghodsi, Fakhri Karray, and Mark Crowley. Stochastic neighbor embedding with gaussian and student-t distributions: Tutorial and survey. *arXiv preprint arXiv:2009.10301*, 2020. URL <https://www.alphaxiv.org/pdf/2009.10301.pdf>.
- [2] Denis C. Ilie-Ablachim, Bogdan Dumitrescu, and Cristian Rusu. Kernel t-distributed stochastic neighbor embedding. *Neurocomputing*, 391:145–155, 2020. doi: 10.1016/j.neucom.2019.12.089.
- [3] Adele Jackson. The mathematics of umap. Technical report, 2019. Preprint. Available at https://adelejackson.com/files/Maths_of_UMAP.pdf.
- [4] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2020. URL <https://arxiv.org/pdf/1802.03426>.
- [5] Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319, 1998. doi: 10.1162/089976698300017467.